

Reflection Write-Up

PaycheckPal — AI-Powered Payslip & Budget Analyzer

Prepared for: Asian Institute of Management — Postgraduate Certificate in Generative AI & Agentic AI

Date: July 2026

What I Built

PaycheckPal is an AI-powered payslip reader and budget planner for Filipino NCR professionals. Users upload or paste a payslip, and Gemini (via the Vercel AI SDK) extracts gross pay, net pay, and the four statutory deductions (SSS, PhilHealth, Pag-IBIG, withholding tax). The app explains each deduction in plain language, flags anomalies against expected contribution rates, and generates a localized 50/30/20 budget recommendation based on net pay and a user-declared monthly remittance. Payslip history is tracked over time in a Supabase-backed, RLS-isolated database that deliberately excludes any personally identifying fields.

Key Learnings

AI provider selection is a moving target, not a one-time decision. I started with the spec's assumption of Anthropic Claude or OpenAI, but pivoted to Google Gemini for free-tier access. Even within Gemini, the specific model mattered: `gemini-2.0-flash` returned a "limit: 0" free-tier quota error on two separate, freshly created API keys, while `gemini-flash-latest` worked immediately. Model availability under the free tier shifts over time — the model name in a tutorial or spec doc from a few months ago is not guaranteed to still be free-tier-eligible.

"It works on my machine" doesn't mean it works in CI or in production, and each environment fails differently for the same root cause. Locally, `next build` succeeded because `.env.local` always had a valid Supabase URL. In GitHub Actions, where no env vars exist at all, the exact same code failed with `Error: supabaseUrl is required.` during static prerendering — a failure mode I'd never hit locally because I'd never actually tested a build with zero environment variables present. I have to double or triple check if the web app is ready for potential production.

Free-tier rate limits are a real product constraint, not just a testing inconvenience.

Gemini's free tier caps at roughly 20 requests/day for the model I used. That's fine for spaced-out real usage, but our own testing during development burned through it repeatedly, and a live demo or grader session could plausibly hit the same wall. This is worth designing around (or budgeting for a paid tier) before treating a free-tier LLM as production-ready. We need to take advantage of the free-tier rate limits to reduce the cost of the Mini-Project's deployment.

Challenges Faced (and How I Diagnosed Them)

- **A pasted-in-the-wrong-field Supabase credential.** `NEXT_PUBLIC_SUPABASE_URL` in `.env.local` was actually a Supabase publishable key value, not a URL — so every Supabase call failed with a generic `Failed to fetch`. I decoded the JWT payload of the (correct) anon key to recover the real project ref and fix the URL. Lesson: when an external service call fails with no useful error, verify the credentials are in the right fields, not just that they're present.
- **Vercel's Deployment Protection silently gated the "live" URL** behind a Vercel login wall, which would have failed the "live app must be publicly accessible" requirement entirely if not caught before submission.
- **An invalid Tailwind class (`bg-gray-750`) rendered as a no-op**, leaving several form inputs with white-on-white text — invisible unless selected. Tailwind fails silently on invalid utility classes rather than erroring, so this shipped without anyone noticing until visually inspected.
- **Supabase's shared free-tier SMTP is unreliable and rate-limited**, which meant the standard email-confirmation signup flow couldn't be trusted for a live demo. I had to explicitly disable "Confirm email" in the Supabase auth settings, and even then, a stray toggle-without-save meant it took two attempts to actually take effect.

What I'd Improve for a Future Version

1. **Server-side range validation using an actual service-role key**, rather than relying on RLS + client-side checks alone, once the app moves beyond MVP scope.
2. **A paid or self-hosted LLM tier** before any real launch, given the free-tier daily quota ceiling observed during development.
3. **Automated visual regression testing** (or at minimum a design-system lint rule) to catch invalid Tailwind classes like `bg-gray-750` before they ship silently.
4. **Actual OCR/image-parsing accuracy testing** against the $\geq 80\%$ success-metric target in the PRD, using a diverse set of real Philippine payslip layouts — something I didn't have time to formally benchmark, only spot-check.
5. **Persisted user preferences beyond remittance** (e.g., a saved default gross-pay range) to reduce repetitive data entry across sessions.

Personal learnings in using AI (Claude Anthropic)

Teamwork with AI is important. When you generate your 7-step ideation, it will help you to specify the scope and problem that you will try to solve. The decision-making process starts from a problem that many people at my age has, then step-by-step de-bottlenecking the problem, then you become clear with the solution through your PRD.

Spec.md. Spec.md is like a recipe book's ingredients, where you must add the actual tools, you used to generate your web app. The actual tools may also change as the project progresses due to software constraints or requirement of subscription (which we don't need for now), so you must re-align the Spec.md with your actual tools used.

AI is your buddy through building the web app. While you ask help from AI to build the app, from generating its code, adding its Supabase (Database), then adding it into GitHub, to deploying it via Vercel App, you must frequently ask for its explanation and basis on why the AI is doing it. You must also ask the AI for help to inspect the files and .md files that it creates so you can quickly check manually if there is an issue.

Test, test, and test. Upon generating your web app, you must check the App from your lens and from the lens of a potential user of the app. Use our advantage of being familiar in different apps to suggest for ways so that the app is as User-Friendly as possible to the potential users. For example, when I tried using the App, the color of the textbox and the text is similar, causing to render the text unreadable, although it has a word in it. Therefore, I ask AI to assist on changing its color.